

Plano Redesenhado: `/implementacao` Skill — Token-Optimized

****Data:**** 27 de agosto de 2026

****Status:**** Plano Aprovado para Implementação

Executivo

Skill `/implementacao` coordena implementação multi-fase com ciclo completo: ****Implementa → Testa → Valida → Verifica → Próxima Fase****. Otimização crítica: máximo ****2 chamadas LLM por fase****, zero loops probabilísticos, gates determinísticos.

1. Separação Nítida: Determinístico vs Probabilístico

Determinístico (Zero Token)

- Orquestração (loops, decisões lógicas)
- Testes (npm test, pytest, etc)
- Type-checking (tsc, mypy)
- Linting (eslint, black, rustfmt)
- Coverage measurement
- Diff & gap classification (script puro)
- Persistência em SQLite
- Relatórios de score

Probabilístico (Token = Mínimo Possível)

- ****Executor:**** 1 chamada LLM por fase (prompt otimizado)
- ****Fixup:**** até 1 chamada se gap é mecânico óbvio
- ****Escalation:**** relatório p/ humano se gap é complexo
- ****MAX:**** 2 chamadas LLM por fase (nunca mais)

2. Fluxo de Execução (Token-Aware)

```
RECEBE
PLANO JSON (estruturado) [Phase 1, Phase 2, Phase 3, ..., Phase N]
↓
PARA
CADA FASE (sequencial)
↓
1.
EXECUTOR (Subagent Fork, 1x chamada LLM) Input: Phase spec + contexto anterior
Output: Mudanças aplicadas (files edited) Cache: Prompt caching ativo para
reutilização
↓
2-5.
VALIDATOR GATES (Script Puro, Zero Token) 2. RUN TESTS npm test / pytest →
JSON report 3. TYPE-CHECK tsc --noEmit → JSON report 4. LINT &
FORMAT eslint / black → JSON report 5. COVERAGE measure vs
threshold → JSON report
↓
6.
AGGREGATE SCORE (Script Puro) • Test pass rate: 0-100% • Type errors: 0-100%
(penalidade por error) • Lint violations: 0-100% (penalidade) • Coverage: 0-100%
(vs target) → FINAL_SCORE = média ponderada
↓
FINAL_SCORE = 100% ?
SIM NÃO ↓ ↓
COMPLETED CLASSIFY GAP Save to DB
Next Phase ↓
Falhas mecânicas? (linting, tipos fáceis)
SIM (óbvio) NÃO (complexo) ↓ ↓
FIXUP-1x ESCALATE (LLM 1x) → Humano
Relatório JSON ↓
RE-VALIDATE (Gates 2-6 again) ↓
SCORE = 100% ? SIM NÃO ↓ ↓
OK FAILED (max retries hit) Save to DB + Alert
```

3. Arquitetura de Arquivos (Token-Efficient)

```
.claude/skills/implementacao/ SKILL.md # Entry + doc completa core/
orchestrator.js # Loop central (0 token, determinístico) executor.js # wrapper p/
subagent fork (1x LLM/fase) fixup-agent.js # wrapper p/ subagent (1x LLM optional)
validators/ (determinístico, zero token) test-runner.js # npm test → JSON
type-checker.js # tsc → JSON linter.js # eslint → JSON
coverage-meter.js # coverage → JSON score-calculator.js # Agrega tudo em score
0-100 gates/ (determinístico, zero token) gap-classifier.js # Analisa falhas
(diff + regex) escalation-detector.js # Decide: fixup automático vs humano
report-generator.js # JSON estruturado p/ escalação db/ state-manager.js #
SQLite: phases, attempts, scores prompts/ executor.prompt.md # Prompt
otimizado p/ geração (cache) fixup.prompt.md # Prompt p/ correção mecânica
```

```
schemas/ ■■■ implementation-plan.json ■■■ validation-report.json ■■■
escalation-report.json
```

4. Economia de Tokens: Mecanismos Específicos

4.1 Executor (1 chamada LLM por fase)

```
// executor.js async function executePhase(phase, previousPhases = []) { // Prompt caching:
contexto anterior reutilizado const cachedContext = await loadPhaseContext(phase.id); const
prompt = ` # Implemente Phase: ${phase.title} ## Spec ${phase.scope.join('\n')} ## Contexto
Anterior (CACHED) ${cachedContext} ## Tests que deve passar ${phase.tests.join('\n')} ##
Output esperado ${phase.expectedFiles.join('\n')} NÃO explique. Apenas modifique os
arquivos. Use este template: \`\`\`file path/to/file.js --- [novo conteúdo] \`\`\`; //
Fork: herdará contexto completo, roda em background const result = await subagent('fork', {
prompt, label: `executor:${phase.id}`, model: 'inherit', }); return result; }
```

****Token savings:****

- ■ Fork herda cache da conversa principal
- ■ Prompt específico, sem fluff
- ■ Uma chamada = terminal (não iterativo)

4.2 Fixup (1 chamada LLM, apenas se gap óbvio)

```
// fixup-agent.js async function attemptFixup(phase, gaps) { // Classifica gaps const
OBVIOUS_GAPS = ['linting', 'formatting', 'type-mismatch-simple']; const isObvious =
gaps.some(g => OBVIOUS_GAPS.includes(g.category) && g.severity === 'low' ); if (!isObvious)
{ // Não tenta fixup automático return null; } const prompt = ` # Corrija gaps simples na
Phase: ${phase.id} ## Gaps detectados ${JSON.stringify(gaps, null, 2)} ## Passos 1. Leia
o erro exato 2. Aplique correção mínima 3. NÃO refatore \`\`\`file [apenas arquivos que
precisam fix] \`\`\`; // Fresh agent (não fork): escalou um nível, mas rápido const
result = await subagent('fresh', { prompt, label: `fixup:${phase.id}`, model: 'inherit',
effort: 'low', // Rápido }); return result; }
```

****Token savings:****

- ■ Apenas chamado se gap é óbvio (filter antes de gastar)
- ■ Prompt ultra-conciso
- ■ effort: 'low' = menos raciocínio

4.3 Gap Classification (Script Puro)

```
// gap-classifier.js function classifyGaps(testReport, typeReport, lintReport,
coverageReport) { const gaps = []; // Test failures if (testReport.failed > 0) {
```

```
gaps.push({ category: 'test-failure', severity: parseTestSeverity(testReport.failures),
count: testReport.failed, fixable: false, // Sempre complexo, requer revisão lógica }); }
// Type errors if (typeReport.errors > 0) { gaps.push({ category: 'type-error', severity:
typeReport.errors > 5 ? 'high' : 'low', count: typeReport.errors, fixable:
typeReport.errors <= 3, // 1-3 erros = tryable }); } // Lint violations if
(lintReport.violations > 0) { gaps.push({ category: 'linting', severity: 'low', count:
lintReport.violations, fixable: true, // Linting é quase sempre fixável }); } // Coverage
if (coverageReport.actual < coverageReport.target) { gaps.push({ category: 'coverage',
severity: coverageReport.gap > 10 ? 'high' : 'low', gap: coverageReport.gap, fixable:
false, // Requer novo código lógico }); } return gaps; } function shouldAttemptFixup(gaps)
{ // Tentamos fixup APENAS se: // 1. Todos os gaps são de categoria baixa/fixável // 2.
Nenhum test failure // 3. Máximo 5 linting issues const hasComplexGaps = gaps.some(g =>
['test-failure', 'coverage'].includes(g.category) || g.severity === 'high' ); if
(hasComplexGaps) return false; const lintCount = gaps .filter(g => g.category ===
'linting') .reduce((sum, g) => sum + g.count, 0); return lintCount <= 5; }
```

****Token savings:****

- ■ Decisão determinística (zero LLM)
- ■ Classifica gaps sem chamar agent
- ■ Escalation automática para humano se complexo

5. Fluxo de Escalation (Humano Entra)

```
// escalation-detector.js async function evaluatePhaseCompletion(phase, score, gaps) { if
(score === 100) { return { status: 'completed', nextPhase: true }; } const canFixup =
shouldAttemptFixup(gaps); if (canFixup) { return { status: 'attempting-fixup', gaps,
willRetry: true }; } // Escalate return { status: 'escalated', score, gaps:
classifyGaps(...reports), recommendation: generateRecommendation(gaps), report:
generateEscalationReport(phase, score, gaps), requiresHumanDecision: true, }; }
```

****Relatório de Escalação (JSON estruturado):****

```
{ "phase": "phase-2", "score": 67, "status": "escalated", "reason": "Test failures +
coverage gap", "gaps": [ { "type": "test-failure", "count": 3, "severity": "high",
"examples": ["test/auth.spec.ts:45", "test/auth.spec.ts:89"], "action": "Revisar lógica de
validação" }, { "type": "coverage", "gap": 12, "severity": "high", "missingBranches":
["error handling", "edge case X"] } ], "nextSteps": [ "1. Revisar falhas de teste", "2.
Adicionar testes para branches faltantes", "3. Reexecutar /implementacao com plano
ajustado" ], "previousAttempts": 1, "maxAttemptsReached": false }
```

6. Loop de Persistência (SQLite, Determinístico)

```
// state-manager.js async function recordPhaseAttempt(phaseId, attempt, score, gaps,
result) { db.run(` INSERT INTO phase_attempts (run_id, phase_id, attempt, score,
```

```
gaps_json, result_status, timestamp) VALUES (?, ?, ?, ?, ?, ?, datetime('now')) \`, [runId,
phaseId, attempt, score, JSON.stringify(gaps), result]); // Atualiza índice da fase
db.run(\` UPDATE phase_status SET current_attempt = ?, last_score = ?, status = ? WHERE
run_id = ? AND phase_id = ? \`, [attempt, score, result, runId, phaseId]); }
```

7. Resumo de Token Economy

Fase	Chamadas LLM	Quando	Token Cost
Executor	1x	Sempre (gera código)	■■■■ (normal)
Fixup (automático)	0-1x	Se gap óbvio + <5 linhas	■ (mínimo)
Escalation	0x	Se gap complexo	Zero
Total/fase	**1-2x**	**Nunca mais**	■ Controlado
5 fases completas	**5-10x**	**No máximo**	■ ~10x tokens

8. Estrutura de Próximos Passos (Redesenhada)

#	Tarefa	Determinístico?	Token Cost	Bloqueia
1	SKILL.md + schemas	Sim	0	2-6
2	orchestrator.js	Sim	0	3-5
3	executor.js (wrapper p/ fork)	Sim	1x/fase	4
4	Validators (test, type, lint)	Sim	0	5
5	gap-classifier.js	Sim	0	6
6	escalation-detector.js	Sim	0	7
7	fixup-agent.js (wrapper, Optional)	Sim	1x se óbvio	8
8	state-manager.js (SQLite)	Sim	0	9
9	Prompts otimizados (cache-friendly)	Sim	0	10
10	Teste end-to-end (5 fases sumário)	Sim	<10x total	■

9. Garantias de Qualidade (Sem Gastar Token)

- ****Gates mecânicos (100% determinísticos):****
- Tests rodam 100% (pass/fail, cobertura)

- Type-check roda 100% (sem supressão)
- Linting roda 100% (formato padrão)
- Scores agregados via fórmula clara

■ ****Escalação inteligente:****

- Gaps complexos → humano decide
- Gaps óbvios → fixup automático (1x LLM máximo)
- Nunca loop infinito

■ ****Rastreabilidade:****

- SQLite persiste cada tentativa
- Relatórios JSON estruturados
- Humano tem visibilidade 100%

Status

****Plano aprovado para implementação imediata.****

Data aprovação: 27 de agosto de 2026